
LoRA on a Budget: Function-Aware Post-Hoc Compression for Adapter Fleets

Anonymous Author(s)

Affiliation

Address

email

Abstract

Low-Rank Adaptation (LoRA) has become a key mechanism for efficiently specializing foundation models into many task- and user-specific variants. Serving many LoRA adapters under a shared resource budget turns post-hoc compression into a two-level allocation problem: rank must be distributed across layers within each adapter and across adapters within the fleet. Although trained LoRAs contain substantial spectral redundancy, spectral reconstruction alone does not determine how much compression an adapter can functionally tolerate. We formulate fleet compression through a functional rank–distortion frontier for each adapter, followed by fleet-level minimax allocation under a shared rank budget. Given exact frontiers, this decomposition is equivalent to joint optimization over all layer ranks. We introduce **Functional Rank Allocation (FRA)**, an unlabeled method for constructing practical frontiers. FRA measures isolated module truncations end to end, uses these measurements as an additive search surrogate solved by dynamic programming, remeasures every complete candidate, and takes the lower-risk envelope with a complementary spectral proposal before exact fleet allocation. Across 78 adapters from three public fleets, FRA consistently improves worst-case and tail utility over uniform spectral compression under constrained budgets. Fixed-budget probes show that isolated costs rank candidates well, but their calibration shifts across proposal shapes; end-to-end verification and the spectral proposal therefore limit functional-search error.

1 Introduction

Foundation models are increasingly adapted into specialized models for a wide range of downstream tasks. Updating every parameter of these models, however, is costly to train, store, and deploy, particularly when many task-specific variants share the same foundation model. Parameter-efficient fine-tuning (PEFT) addresses this cost by learning only a small set of task-specific parameters while leaving the pretrained model frozen. Low-rank adaptation (LoRA) [Hu et al., 2022] has emerged as an especially effective form of PEFT. Rather than modifying a weight matrix directly, LoRA learns an additive update represented as the product of two low-rank factors. This parameterization can retain the benefits of task-specific fine-tuning with far fewer trainable and stored parameters, making each specialization a compact adapter that can be shared, composed with a common base model, and loaded on demand.

Modern inference frameworks increasingly support serving many LoRA adapters from a shared base model, with adapters selected or loaded on a per-request basis. Systems such as vLLM [Kwon et al., 2023], S-LoRA [Sheng et al., 2024], Punica [Chen et al., 2024], and LoRAX [Predibase, 2023] enable such multi-LoRA serving by batching, routing, and managing adapters around shared base weights. At this scale, adapters compete for finite GPU cache capacity, transfer bandwidth, and rank-dependent inference resources. Compress-then-Serve shows that compressing an adapter fleet can translate into substantial serving gains [Brüel-Gabrielsson et al., 2025].

There is substantial capacity to reclaim from trained LoRA adapters. To characterize rank redundancy in the wild, we analyze 96,370 matrices from 200 popular LoRA repositories on Hugging Face and find that spectral energy is strongly concentrated: at $\tau = .90$, the median layer retains only 31.2% of its nominal rank (Fig. 1). This widespread redundancy makes post-hoc compression an attractive way to reduce adapter memory, transfer volume, and rank-dependent serving overhead, allowing a fixed serving capacity to accommodate a larger adapter fleet [Vasani et al., 2025, Kumaravelu et al., 2026, Brüel-Gabrielsson et al., 2025, He et al., 2026].

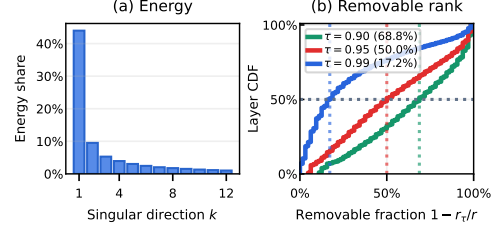


Figure 1: **LoRA fleets contain substantial compressible rank.** Among 96,370 matrices, the median layer retains only 31.2% of rank at $\tau = .90$.

The challenge, however, is no longer simply whether rank can be removed, but how a limited rank budget should be allocated. Within a single adapter, different layers need not tolerate the same degree of truncation. Accordingly, the available rank should be distributed to preserve the adapter’s behavior as well as possible. At fleet scale, the problem becomes coupled, as adapters share the same serving capacity yet may differ substantially in how much compression they can tolerate. Effective post-hoc compression must therefore decide both how rank is divided across layers and how the shared budget is divided across adapters.

We address this two-level allocation problem with **Functional Rank Allocation (FRA)**. For each adapter, the ideal objective is a functional rank–distortion frontier that captures the minimum behavioral change achievable at every retained-rank budget. Constructing this frontier exactly is combinatorial, so FRA uses end-to-end measurements of individual module truncations to form an additive search surrogate and solves the resulting rank-allocation problem exactly by dynamic programming. Each proposed allocation is then instantiated and remeasured end to end, so the surrogate is used only to generate candidates rather than to assign their final risk. Because the additive surrogate can be calibrated differently across allocation shapes, FRA also evaluates a complementary spectral candidate and retains the lower-risk allocation at each budget. The resulting measured frontiers are finally coupled through an exact fleet-level minimax allocation under the shared rank budget. We summarize our contribution as follows:

- We formulate post-hoc LoRA fleet compression as a two-level resource allocation problem: each adapter first defines a functional rank–distortion frontier over its layer-wise allocations, after which the shared fleet budget is allocated across these frontiers. Given exact frontiers, this decomposition is equivalent to joint optimization over all layer ranks and introduces no additional approximation.
- We introduce **Functional Rank Allocation (FRA)**, an unlabeled method for constructing practical functional frontiers. FRA uses isolated end-to-end truncation costs to search for layer-rank allocations via dynamic programming, remeasures every complete candidate before it enters the frontier, and adds a spectral candidate to guard against cross-proposal surrogate miscalibration.
- We evaluate FRA across 78 adapters from three public fleets and show that function-aware allocation substantially improves tail utility under constrained budgets. We show that isolated costs preserve within-budget ordering, while proposal-shape-dependent calibration error explains why the complementary spectral candidate remains valuable.

2 Background

Post-Hoc LoRA Compression For a pretrained weight matrix $W_{0,\ell} \in \mathbb{R}^{d_{\text{out}} \times d_{\text{in}}}$, LoRA keeps $W_{0,\ell}$ frozen and learns an additive low-rank update $\Delta W_{i,\ell} = \frac{\alpha_{i,\ell}}{r_{i,\ell}} B_{i,\ell} A_{i,\ell}$, where $B_{i,\ell} \in \mathbb{R}^{d_{\text{out}} \times r_{i,\ell}}$, $A_{i,\ell} \in \mathbb{R}^{r_{i,\ell} \times d_{\text{in}}}$, and $r_{i,\ell}$ is the nominal rank [Hu et al., 2022]. An adapter consists of such updates across a collection of modules while sharing the same frozen base model. An already-trained LoRA module can be further compressed without retraining by reducing the rank of its effective update $\Delta W_{i,\ell}$. Let its compact singular value decomposition be $\Delta W_{i,\ell} = U_{i,\ell} \Sigma_{i,\ell} V_{i,\ell}^\top$, with singular values $\sigma_{i,\ell,1} \geq \dots \geq \sigma_{i,\ell,r_{i,\ell}}$. Retaining the leading k singular directions gives $\Delta W_{i,\ell,k} = U_{i,\ell,:k} \Sigma_{i,\ell,:k} V_{i,\ell,:k}^\top$, the optimal rank- k approximation under the Frobenius norm. The truncated

update can be refactored into rank- k LoRA factors and served in the same form as the original module. Such SVD recompression is used in post-hoc adapter compression [Brüel-Gabrielsson et al., 2025, Kumaravelu et al., 2026]. It can also be computed directly from low-rank factors or compact coordinate matrices without materializing the full update, substantially reducing the recompression cost [Kumaravelu et al., 2026, Li and Voigt, 2026]. A common spectral compression rule selects the smallest retained rank whose leading singular directions preserve a prescribed fraction τ of the spectral energy, $k_{i,\ell}(\tau) = \min \left\{ k \in \{1, \dots, r_{i,\ell}\} : \frac{\sum_{j=1}^k \sigma_{i,\ell,j}^2}{\sum_{j=1}^{r_{i,\ell}} \sigma_{i,\ell,j}^2} \geq \tau \right\}$. We additionally allow $k = 0$, which removes the corresponding LoRA branch entirely.

LoRA Fleets and Serving Budgets Modern serving systems can attach many independently trained LoRA adapters to a common base model and dynamically load or batch them across requests, as in S-LoRA [Sheng et al., 2024], Punica [Chen et al., 2024], and LoRAX [Predibase, 2023]. We refer to such a collection of N adapters as a *LoRA fleet*. For adapter i , let $k_{i,\ell}$ denote the retained rank of module ℓ after post-hoc compression and $K_i = \sum_{\ell} k_{i,\ell}$ its aggregate retained rank. Within each fleet, architecture and module shapes are fixed across adapters. We report a hardware-agnostic budget in retained rank-direction slots and impose $\sum_{i=1}^N K_i \leq B$, where B is a direction budget rather than an exact byte count. Modules of different width have different per-direction storage cost; a deployment can replace $k_{i,\ell}$ by $c_{\ell} k_{i,\ell}$ without changing either optimizer.

3 Method

3.1 Problem Formulation

Given the fleet setting above, post-hoc LoRA compression requires deciding how capacity should be distributed across layers within each adapter and across adapters sharing the same serving capacity. We formulate both allocation problems under a common functional risk that quantifies how compression changes model behavior.

Single-Adapter Compression. Consider adapter i with L_i LoRA modules and layer-wise rank allocation $\mathbf{k}_i = (k_{i,1}, \dots, k_{i,L_i})$, where $k_{i,\ell} \in \{0, \dots, r_{i,\ell}\}$ and $k_{i,\ell} = 0$ removes that LoRA branch from execution. Let f_i denote the uncompressed adapter and $f_{i,\mathbf{k}_i}$ its compressed counterpart. We define $R_i(\mathbf{k}_i) = \mathbb{E}_{x \sim \mathcal{C}_i} [\phi(f_i(x), f_{i,\mathbf{k}_i}(x))]$, where $\mathcal{C}_i$ is a small unlabeled calibration set and ϕ measures behavioral change. By default, we use output-distribution Jensen–Shannon (JS) divergence. This is an unlabeled behavioral proxy, not a guarantee on downstream utility, which we evaluate separately on held-out labeled data. For an adapter-level rank budget K , the ideal compression defines the functional rank–distortion frontier

$$R_i^*(K) = \min_{\|\mathbf{k}_i\|_1 \leq K} R_i(\mathbf{k}_i), \quad \mathbf{k}_i^*(K) \in \arg \min_{\|\mathbf{k}_i\|_1 \leq K} R_i(\mathbf{k}_i). \quad (1)$$

Multi-Adapter Compression. For a fleet of N adapters sharing a total rank budget B , we allocate an adapter-level budget K_i to each adapter by solving

$$\min_{K_1, \dots, K_N} \max_{i=1, \dots, N} R_i^*(K_i) \quad \text{s.t.} \quad \sum_{i=1}^N K_i \leq B. \quad (2)$$

Note that we use the minimax objective to prevent a small subset of adapters from absorbing disproportionate compression damage. Other objectives, such as mean or workload-weighted risk, can be optimized over the same frontiers. Importantly, this two-level decomposition is exact: given the exact frontiers $R_i^*(K)$, Eq. (2) is equivalent to jointly optimizing all layer ranks $\{k_{i,\ell}\}$ under the global budget.

3.2 Functional Rank Allocation

Constructing the exact frontier $R_i^*(K)$, however, requires searching over a combinatorial space of layer-wise rank allocations. FRA approximates this search using functional measurements of individual truncation actions, while remeasuring each complete candidate end to end. We probe each compression action in isolation. For module ℓ , let $C_{i,\ell}(k) = R_i(\mathbf{k}_i^{(\ell \rightarrow k)})$ denote the functional risk

when only module ℓ is truncated to rank k and all other modules remain uncompressed. Each $C_{i,\ell}(k)$ is an end-to-end functional measurement on unlabeled calibration prompts, rather than a spectral or activation-based proxy.

We use the sum of isolated costs only as an additive search surrogate, $\tilde{R}_i(\mathbf{k}_i) = \sum_{\ell=1}^{L_i} C_{i,\ell}(k_{i,\ell})$. For budget K , we solve

$$\hat{\mathbf{k}}_i^F(K) = \arg \min_{\|\mathbf{k}_i\|_1 \leq K} \sum_{\ell=1}^{L_i} C_{i,\ell}(k_{i,\ell}). \quad (3)$$

Eq. (3) is a multiple-choice knapsack problem, where each module forms one choice group and the retained rank is its resource cost. We solve it exactly for the surrogate objective by dynamic programming in $O(L_i K r_{\max})$ time, where $r_{\max}$ is the largest candidate rank.

The additive surrogate is used only to search for a rank allocation. For every proposed $\hat{\mathbf{k}}_i^F(K)$, we instantiate the complete compressed adapter and remeasure its end-to-end functional risk, $\hat{R}_i^F(K) = R_i(\hat{\mathbf{k}}_i^F(K))$. Thus, approximation errors affect which allocation is proposed, but not the risk value exposed to the fleet-level optimizer. The surrogate can incur search error when isolated effects interact. We retain the composition-free spectral proposal $\hat{\mathbf{k}}_i^S(K) \in \arg \min_{\|\mathbf{k}\|_1 \leq K} \sum_{\ell} \|\Delta W_{i,\ell} - \Delta W_{i,\ell,k_\ell}\|_F^2$, equivalently keeping the largest prefix-constrained singular energies. We evaluate it with the same end-to-end risk, $\hat{R}_i^S(K) = R_i(\hat{\mathbf{k}}_i^S(K))$, and retain the lower measured risk at each budget, i.e., $\hat{R}_i(K) = \min\{\hat{R}_i^F(K), \hat{R}_i^S(K)\}$. We denote the corresponding retained rank vector by $\hat{\mathbf{k}}_i(K)$. This measured curve approximates the ideal frontier $R_i^*(K)$.

3.3 Fleet-Level Minimax Allocation

After Sec. 3.2, each adapter i is represented by a measured functional frontier $\hat{R}_i(K)$ together with the corresponding layer-rank allocation. Because the global constraint is an upper budget, we take the monotone closure $\bar{R}_i(K) = \min_{K' \leq K} \hat{R}_i(K')$: if a smaller allocation achieves lower risk, there is no reason to spend additional rank. The corresponding rank vector is retained for deployment. Given these frontiers, we solve

$$\min_{K_1, \dots, K_N} \max_i \bar{R}_i(K_i) \quad \text{s.t.} \quad \sum_{i=1}^N K_i \leq B. \quad (4)$$

For any candidate risk level τ , adapter i requires at least $K_i(\tau) = \min\{K : \bar{R}_i(K) \leq \tau\}$ rank to remain below τ . Thus, τ is feasible if and only if $\sum_i K_i(\tau) \leq B$. We enumerate the finite set of measured risk levels and select the smallest feasible τ , yielding the exact minimax solution over the constructed frontiers. Among allocations attaining the same minimax value, we minimize total risk and then maximize budget utilization.

4 Experiments

We evaluate FRA and competing allocation rules across three diverse multi-adapter fleets. **LoRA Land** ($N = 12$, Mistral-7B [Zhao et al., 2024a]) contains 12 reasoning, classification, and generation tasks, including GSM8K, WikiSQL, and DBpedia, scored with standard task-specific metrics such as accuracy, BLEU, and Rouge; **Lots-of-LoRAs** ($N = 25$, Mistral-7B-Instruct-v0.2 [Brüel-Gabrielsson et al., 2024]) contains 25 specialized domain adapters—of the 33 controlled-pool members, three have no public evaluation dataset and five leave a held-out split too small to score, so 25 is the full reachable pool rather than a sample; and **LoRARetriever** ($N = 41$, Llama-2-7B [Zhao et al., 2024b]) contains 41 classification and GLUE/SuperGLUE adapters.

Metric and budgets. We report retention $R \equiv \frac{M(\text{compressed}) - M(\text{base})}{M(\text{uncompressed}) - M(\text{base})}$ against the un-adapted model, so $R = 1$ means compression cost nothing and $R \leq 0$ means the compressed adapter is no better than serving no adapter at all. Fleet budgets are set by what uniform truncation spends at a given threshold, giving three regimes: **moderate** ($\tau_{\text{uni}} = .90$), **binding** (.70) and **severe** (.50). Table 1 reports each fleet’s full rank, the resulting budgets, and what they cost in parameters. The

Table 1: **Fleet scale and what each budget costs.** N is the number of adapters in each pool and r is the nominal rank. Grouped-query attention makes a q direction cost 8,192 parameters against 5,120 for k/v , so on the Mistral fleets the parameter share departs from the rank share.

Pool				Metric	Full rank	Moderate ($\tau_{\text{uni}}=.90$)	Binding ($\tau_{\text{uni}}=.70$)	Severe ($\tau_{\text{uni}}=.50$)
Fleet	r	N	Modules					
LoRA Land	8	12	$64 \times (q,v)$	Directions	6,144	2,288 (37.2%)	1,320 (21.5%)	936 (15.2%)
				Params (M)	40.89	15.28 (37.4%)	8.70 (21.3%)	6.09 (14.9%)
Lots-of-LoRAs	16	25	$96 \times (q,k,v)$	Directions	38,400	19,200 (50.0%)	8,800 (22.9%)	4,912 (12.8%)
				Params (M)	235.93	119.59 (50.7%)	55.57 (23.6%)	30.50 (12.9%)
LoRARetriever	8	41	$64 \times (q,v)$	Directions	20,992	5,880 (28.0%)	3,184 (15.2%)	2,680 (12.8%)
				Params (M)	171.97	48.17 (28.0%)	26.08 (15.2%)	21.95 (12.8%)

Table 2: **Nested ablation of the FRA design at matched fleet budgets.** Uniform $\rightarrow$ Intra-Spec $\rightarrow$ E2E-Spec $\rightarrow$ FuncDP $\rightarrow$ FRA progressively adds within-adapter reallocation, cross-adapter reallocation, functional inner search, and the spectral safeguard. The *Candidate Oracle* gives, per statistic, the best value any allocation in the union of the five families reaches at that budget, so a block’s entries need not share one allocation. FRA is bold; green marks the best deployable entry.

Pool	Method	Probes	Moderate ($\tau_{\text{uni}}=.90$)			Binding ($\tau_{\text{uni}}=.70$)			Severe ($\tau_{\text{uni}}=.50$)		
			Mean	p_{10}	Worst	Mean	p_{10}	Worst	Mean	p_{10}	Worst
LoRA Land	<i>Cand. Oracle</i>	<i>labeled</i>	<i>1.009</i>	<i>0.990</i>	<i>0.966</i>	<i>0.983</i>	<i>0.949</i>	<i>0.879</i>	<i>0.921</i>	<i>0.878</i>	<i>0.672</i>
	Uniform	$\times$	0.913	0.873	0.081	0.725	0.010	-0.293	0.748	0.047	-0.015
	Intra-Spec	$\times$	0.895	0.898	0.036	0.764	0.185	0.000	0.554	-0.011	-0.414
	E2E-Spec	curve	0.974	0.920	0.897	0.829	0.640	0.203	0.530	-0.030	-1.309
	FuncDP	curve + layer	0.985	0.949	0.931	0.963	0.915	0.879	0.907	0.692	0.620
	FRA	$2 \times \text{curve} + \text{layer}$	0.985	0.949	0.931	0.963	0.915	0.879	0.907	0.692	0.620
Lots-of-LoRAs	<i>Cand. Oracle</i>	<i>labeled</i>	<i>1.033</i>	<i>1.000</i>	<i>1.000</i>	<i>1.017</i>	<i>0.974</i>	<i>0.955</i>	<i>0.998</i>	<i>0.923</i>	<i>0.877</i>
	Uniform	$\times$	0.989	0.960	0.877	0.968	0.885	0.542	0.921	0.797	0.351
	Intra-Spec	$\times$	0.998	0.954	0.927	0.961	0.882	0.515	0.892	0.469	0.321
	E2E-Spec	curve	0.998	0.977	0.895	0.974	0.915	0.599	0.961	0.879	0.740
	FuncDP	curve + layer	1.004	0.988	0.910	0.989	0.934	0.800	0.973	0.912	0.599
	FRA	$2 \times \text{curve} + \text{layer}$	1.000	0.982	0.935	1.000	0.949	0.910	0.971	0.912	0.599
LoRARetriever	<i>Cand. Oracle</i>	<i>labeled</i>	<i>0.986</i>	<i>0.897</i>	<i>0.750</i>	<i>0.933</i>	<i>0.784</i>	<i>0.500</i>	<i>0.874</i>	<i>0.667</i>	<i>0.250</i>
	Uniform	$\times$	0.905	0.714	0.500	0.819	0.500	0.250	0.791	0.500	0.250
	Intra-Spec	$\times$	0.948	0.818	0.600	0.840	0.571	0.143	0.776	0.364	0.143
	E2E-Spec	curve	0.930	0.818	0.500	0.857	0.680	0.500	0.780	0.500	0.250
	FuncDP	curve + layer	0.906	0.667	0.500	0.833	0.600	0.250	0.822	0.600	0.250
	FRA	$2 \times \text{curve} + \text{layer}$	0.933	0.800	0.500	0.894	0.727	0.500	0.829	0.600	0.250

severe budget leaves only one feasible solution for the threshold-grid component. Methods with arbitrary-rank candidates retain allocation freedom, making the severe column the cleanest separation.

Calibration and its cost. No rule except the Candidate Oracle sees a downstream label. Calibration prompts are disjoint from the scored examples in every pool (LoRA Land: examples 200–215 calibrate, 0–199 score; Lots-of-LoRAs: train rows 0–15 calibrate, the held-out split scores; LoRARetriever: eval-set rows 0–15 calibrate, 16–49 score). What differs is how many unlabeled forward passes each rule needs per adapter (Table 3). Here $|\mathcal{G}|$ is the number of budgets on the measured grid: 65 for LoRA Land, 97 for Lots-of-LoRAs, and 65 for LoRARetriever. The first two rungs need no forward pass at all – they are decided from the spectra – so every gain they show is free. FRA’s Lr single-layer probes (512 on LoRA Land, with $L=64$ modules and $r=8$) dominate its cost, but are paid once per adapter at registration and reused at every budget; on one GH200 that is about 8 minutes per adapter, and nothing is added to the serving path.

Measured Functional Frontiers Matter. Spectral information can propose a layer-rank allocation, but the spectrum alone does not provide the end-to-end risk value required by the fleet allocator. This distinction matters at fleet scale, where the outer allocator compares adapters through their frontier values rather than through their spectra. We therefore separate proposal generation from

Table 3: **Calibration cost per adapter.** Unlabeled forward passes each rule needs, all offline at registration time and none on the serving path.

Rule	Passes	What they measure
Uniform	0	—
Intra-Spec	0	—
E2E-Spec	$ \mathcal{G} $	each spectral proposal
FuncDP	$Lr + \mathcal{G} $	ranks 0: $r-1$, then proposals
FRA	$Lr + 2 \mathcal{G} $	both proposal families

frontier evaluation: spectral and functional search mechanisms propose candidate allocations, but every complete candidate is remeasured end to end before entering the fleet optimizer.

Ablating the FRA Design. Table 2 relaxes the restrictions of uniform spectral compression one rung at a time at a fixed fleet budget: Intra-Spec redistributes rank across modules by spectral importance, E2E-Spec additionally releases the adapter-level budgets and allocates them by exact minimax over measured end-to-end frontiers, FuncDP replaces the spectral inner allocation with the functional search of Sec. 3.2, and FRA takes the lower-risk envelope of the two proposal families. From Intra-Spec onward modules may receive rank zero, and the Candidate Oracle is a reference over the same candidate families rather than a rung. Within-adapter spectral reallocation alone is inconsistent, whereas releasing adapter-level budgets helps more when compression sensitivity differs across tenants. At the binding budget on LoRA Land, worst-case retention rises from -0.293 under Uniform to 0.203 with E2E-Spec, and functional inner allocation further raises it to 0.879 ; on Lots-of-LoRAs, FuncDP improves the corresponding worst case from 0.599 to 0.800 . The spectral envelope then protects against failures of the functional surrogate: FRA preserves the 0.879 worst case on LoRA Land, raises Lots-of-LoRAs from 0.800 to 0.910 , and restores LoRARetriever from 0.250 to 0.500 . FRA weakly dominates either proposal family in measured functional risk at each constructed adapter budget, while Table 2 evaluates downstream utility on held-out data.

When Does Inner Allocation Help? The dynamic programming algorithm solves the surrogate optimization problem in Eq. (3) exactly. To assess how well the surrogate ranks candidate allocations, we measure within-budget Spearman rank correlation between surrogate and measured risks. Let $\mathcal{A}_i(K) = \{\mathbf{k} : \sum_{\ell} c_{\ell} k_{\ell} = K\}$ denote the set of candidate allocations for adapter i with parameter budget K . We define ranking fidelity as $\rho_i(K) = \text{corr}_S(\{\tilde{R}_i(\mathbf{k})\}_{\mathbf{k} \in \mathcal{A}_i(K)}, \{R_i(\mathbf{k})\}_{\mathbf{k} \in \mathcal{A}_i(K)})$, where corr_S denotes Spearman’s rank correlation coefficient. Across budgets, the resulting correlations average 0.638 , 0.629 , and 0.852 across the three pools, respectively (Fig. 2a): LoRARetriever is the most, not least, rank-faithful. The failure instead appears across proposal shapes. Matching functional and spectral proposals to within 2% parameter cost (excluding identical allocations), measurement favors functional search in 73.7%, 58.0%, and 28.3% of 585, 2,038, and 1,770 comparisons. More independently, the median measured ratio favors spectral for 1/12, 9/25, and 39/41 adapters. Median surrogate/measurement spectral-to-functional risk ratios are $2.74/1.73$, $3.02/1.15$, and $1.76/0.56$: miscalibration grows from $1.6\times$ to $3.1\times$ and reverses the measured sign on LoRARetriever (Fig. 2b). This proposal-shape-dependent surrogate miscalibration is consistent with shape-dependent cross-layer cancellation, but does not identify it over higher-order interactions, allocation-severity effects, or selection-induced optimism. End-to-end remeasurement and the spectral envelope limit this error.

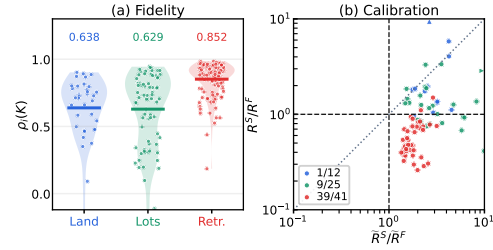


Figure 2: **Ranking fidelity does not explain the cross-proposal reversal.** (a) fixed-budget Spearman fidelity, fleet means above. (b) per-adapter median surrogate versus measured risk ratio on a shared 10^{-1} – 10^1 scale, colors as in (a); the legend counts spectral-favoring adapters and triangles mark the two adapters beyond the axes.

5 Conclusion

We presented FRA, a function-aware post-hoc compression framework for allocating limited rank across layers and adapters in LoRA fleets. FRA constructs measured functional rank–distortion frontiers through isolated truncation probes, dynamic programming, and end-to-end verification, then performs exact fleet-level minimax allocation under a shared budget. Experiments across three public adapter fleets demonstrate that FRA achieves favorable trade-offs between rank reduction and downstream utility across a range of compression budgets, making large-scale adapter serving more resource-efficient and resilient to heterogeneous compression sensitivity.

References

- Rickard Brüel-Gabrielsson, Jiacheng Zhu, Onkar Bhardwaj, Leshem Choshen, Kristjan Greenewald, Mikhail Yurochkin, and Justin Solomon. The Lots-of-LoRAs adapter collection. <https://huggingface.co/Lots-of-LoRAs>, 2024. Hugging Face model collection released with Brüel-Gabrielsson et al. [2025].
- Rickard Brüel-Gabrielsson, Jiacheng Zhu, Onkar Bhardwaj, Leshem Choshen, Kristjan Greenewald, Mikhail Yurochkin, and Justin Solomon. Compress then serve: Serving thousands of LoRA adapters with little overhead. In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*. PMLR, 2025.
- Lequn Chen, Zihao Ye, Yongji Wu, Danyang Zhuo, Luis Ceze, and Arvind Krishnamurthy. Punica: Multi-tenant LoRA serving. In *Proceedings of Machine Learning and Systems (MLSys)*, 2024.
- Zhengbao He, Ruiqi Ding, Zhehao Huang, Ruikai Yang, Tao Li, and Xiaolin Huang. Compress then merge: From multiple LoRAs into one low-rank adapter. *arXiv preprint arXiv:2606.03723*, 2026.
- Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations*, 2022. URL <https://openreview.net/forum?id=nZeVKeeFYf9>.
- Vishnuprasadh Kumaravelu, Sunil Gupta, and P. K. Srijith. Post-optimization adaptive rank allocation for LoRA. *arXiv preprint arXiv:2604.27796*, 2026.
- Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles*, pages 611–626, 2023.
- Shenghui Li and Thiemo Voigt. FraQ: Efficient coordinate-space recompression for federated low-rank adaptation. *arXiv preprint arXiv:2608.03605*, 2026.
- Predibase. LoRAX: Multi-LoRA inference server that scales to 1000s of fine-tuned LLMs. GitHub repository, 2023. URL <https://github.com/predibase/lorax>. Accessed 2 September 2026.
- Ying Sheng, Shiyi Cao, Dacheng Li, Banghua Zhu, Zhuohan Li, Danyang Zhuo, Joseph E. Gonzalez, and Ion Stoica. S-LoRA: Serving thousands of concurrent LoRA adapters. In *Proceedings of Machine Learning and Systems (MLSys)*, 2024.
- Bhoomit Vasani, Jack FitzGerald, Anjie Fang, and Sushmit Vaish. PHLORA: Data-free post-hoc low-rank adapter extraction from full-rank checkpoint. *arXiv preprint arXiv:2509.10971*, 2025.
- Justin Zhao, Timothy Wang, Wael Abid, Geoffrey Angus, Arnav Garg, Jeffery Kinnison, Alex Sherstinsky, Piero Molino, Travis Addair, and Devvret Rishi. Lora land: 310 fine-tuned LLMs that rival GPT-4, a technical report, 2024a.
- Ziyu Zhao, Leilei Gan, Guoyin Wang, Wangchunshu Zhou, Hongxia Yang, Kun Kuang, and Fei Wu. LoraRetriever: Input-aware LoRA retrieval and composition for mixed tasks in the wild. In *Findings of the Association for Computational Linguistics: ACL 2024*, 2024b.